

C in Assembly Language

Chapter Contents

Overview	134
Motivation	134
Software Development Tools	135
Program Build Tools	135
Compiler	136
Assembler	136
Linker/Loader	139
Programmer	139
Debugger	140
C Language Fundamentals	140
Program and Functions	140
Start-Up Code	141
Types of Memory	141
A Program's Memory Requirements	142
Making Functions	143
Register Use Conventions	144
Function Arguments	144
Function Return Value	145
Prolog and Epilog	145
Prolog	145
Epilog	146
Exception Handlers	147
Controlling the Program's Flow	148
Conditionals	148
If/Else	148
Switch	150
Loops	152
Do While	152
While	153
For	155
Calling Subroutines	156

Accessing Data in Memory	157
Statically Allocated Memory	157
Automatically Allocated Memory	158
Dynamically Allocated Memory and Other Pointers	159
Array Elements	159
Summary	162
Exercises	162
References	164

Overview

In Chapter 4 we examined the Arm Cortex-M0 processor core, including the instructions that it can execute, the registers for general-purpose data processing, and methods to access memory. In this chapter we will see the object code that the compiler creates to implement the C-language source code. Understanding this object code will help us debug our programs and write source code that is more efficient in its use of time or memory.

Motivation

We have seen the instructions that the Arm Cortex-M0 processor core can execute. Each instruction performs simple operations, such as adding two integers, comparing values, or pushing registers onto the stack in memory. Creating a program using these instructions requires the developer to make many small decisions: where to place a data variable in memory, which register to use to process that variable, which kind of conditional branch to use after a comparison, how to share data between functions, and so forth. This complexity slows software development and introduces many opportunities for errors.

High-level programming languages were developed to free the software developer from these issues. C and C++ are the dominant programming languages for embedded systems. We use a language translation tool called a compiler to convert an application program from one high-level language to assembly language, and another tool called an assembler to convert the assembly language to object code. The CPU executes the program by reading each object code instruction and then executing it very quickly (e.g. within one or two CPU clock cycles).

In some programming languages (e.g. Java, Perl, Python), an application program is not converted to the CPU's native executable format. Instead, at runtime the application program is processed by another program (such as an interpreter or virtual machine) to carry out the work specified. Java code can be compiled to an intermediate form called bytecode before the program is run. At runtime the virtual machine program interprets and executes the bytecode. The interpreter or virtual machine program increases the amount of time and memory needed to run an application program. Compiled languages dominate embedded systems development because they do not incur these runtime and memory overheads, and also because they offer more predictable behavior.

A **toolchain** is a set of tools used to build the program (to convert it from source code to executable object code), download it to the MCU's memory, and control its execution for debugging.

The toolchain must be configured to support the specific type of processor we are using and how much memory is in the system.

Using a high-level programming language introduces trade-offs. We expect the toolchain to generate an object code that is correct, and also reasonably fast and small. It may be possible to manually create faster or smaller object code by writing the assembly code ourselves, but we will do this only when necessary.

Software Development Tools

Three major types of tools used for embedded software development are the **program build toolchain**, the **programmer**, and the **debugger**. The program build toolchain translates a program into a format that the MCU can understand, stored in an executable file. The programmer programs that information into the MCU's program memory. This memory is nonvolatile (typically Flash ROM), so it will remain even after power is removed. The debugger enables the developer to control program execution and examine the program state (e.g. current instruction, values of processor registers, and data memory) as it runs on the processor.

These tools are often grouped together in a single integrated development environment (IDE) to simplify development. In this textbook we cover the Keil μ Vision (microVision) IDE.

Program Build Tools

Figure 5.1 gives an overview of the build tools and the files they process to create the final executable file. Build tool manuals are available online [1]. Programs are built of modules that are translated through a series of formats and then combined into an executable file. The file `my_module.c` contains the module's code in the C language format. This is compiled to create an assembly language module in the file `my_module.s`, which is then assembled to create an object module in the file `my_module.o`. This module is linked with other modules (also in object format) to create the executable file.

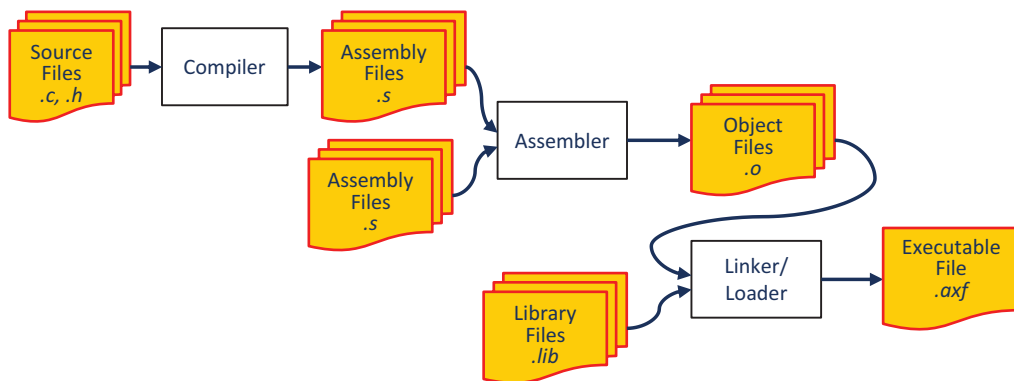


Figure 5.1 Program build tools process different files to create a file holding the executable program.

Compiler

A compiler converts a module from a high-level language such as C into assembly language. Keil μ Vision uses the armcc compiler [2]. The compiler compiles one source file (e.g. .c) at a time, following these steps:

- Pre-process the .c file by including the text of any included files (e.g. with `#include`) and performing macro substitution (e.g. `#define`).
- Verify that the syntax of the program is correct.
- Create intermediate code from the source code.
- Perform high-level optimization of the intermediate code.
- Generate assembly code from the intermediate code and allocate variable uses to registers.
- Perform low-level optimization of the assembly code.
- Generate an output file (.s) containing the assembly code for the module.

Note that the compilation process is not one-to-one. The compiler's primary goal is to create object code that **correctly implements the requirements** specified by the source code. This means that there are many possible correct object code versions of a single C language program. They may differ in the number and type of instructions used, which registers are used, how much data memory is used, how code is laid out, and other aspects as well.

A compiler can generate an assembly language listing file to help understand the code it has generated. The next section has an example in Listing 5.2. The professional version of the Keil MDK-ARM IDE can generate such a listing, but the light version cannot.

A source file may refer to variables and functions defined in other files using external references. Since at this point in the build process the addresses of these variables and functions haven't been defined yet, the output assembly code file uses the symbols (text names) as placeholders.

Commonly used functions are often gathered together in a library to simplify their reuse. The C language defines many standard libraries, including common mathematical functions (`math`), input and output (`stdio`), string processing (`string`), time and date (`time`).

Assembler

An assembler converts a human-readable assembly language module into an object module that describes the size and contents of the memory sections required for the module. Each memory section may hold instructions, data, or both. The assembler processes one .s file at a time. This file may have been written directly by a developer or created by a compiler. Keil μ Vision uses the armasm assembler [3].

The assembler steps through the input file one line at a time, though it may make multiple passes through the file. It uses a location counter for each memory section to track where to place (or allocate) the current memory item, and then updates the counter accordingly.

```

Stack_Size      EQU      0x00000400
Stack_Mem       AREA     STACK, NOINIT, READWRITE, ALIGN=3
__initial_sp
; Vector Table Mapped to Address 0 at Reset
                AREA     RESET, DATA, READONLY
                EXPORT   __Vectors
                EXPORT   __Vectors_End
                EXPORT   __Vectors_Size
__Vectors       DCD      __initial_sp          ; Top of Stack
                DCD      Reset_Handler        ; Reset Handler
                DCD      NMI_Handler          ; NMI Handler
                DCD      HardFault_Handler    ; Hard Fault Handler

; More vectors here
                AREA     |.text|, CODE, READONLY
; Reset handler routine
Reset_Handler   PROC
                EXPORT   Reset_Handler        [WEAK]
                IMPORT   __main
                IMPORT   SystemInit
                LDR      R0, =SystemInit
                BLX      R0
                LDR      R0, =__main
                BX       R0
                ENDP

```

Listing 5.1 Example of assembly language file (filename suffix is .s).

Listing 5.1 shows part of an assembly language file. An assembly language file is made of several types of elements.

An **instruction** consists of a mnemonic and possibly operands, as described in the previous chapter. For example, upon seeing instruction **BLX R0**, the assembler will generate the encoded Thumb instruction according to the Arm architecture specification [4] and advance the location counter. The **BLX R0** instruction will be encoded as the 16-bit value 0x4780:

- Bits 15 through 7 (the most-significant bits) are set to 0100 0111 1 to specify the **BLX** instruction.
- Bits 6 through 3 are cleared to 0000 to specify register R0.
- Bits 2 through 0 are unused and cleared to 000.

A **directive** directs the assembler to do something.

- **AREA** tells the assembler to place the next items declared in a given memory area and change to that area's location counter.
- **SPACE** tells the assembler to allocate space for data but not fill it with any specific data, and then advance the current location counter.
- Define constant data (**DCD**) tells the assembler to define a 32-bit constant data item, placing that data in the next available memory location.
- **EQU** tells the assembler to define a temporary symbolic name and assign it a specific value, but not allocate any memory for it.
- Other common assembler directives are **IMPORT**, **EXPORT**, **THUMB**, **PROC**, **ENDP**, and **DCB**.

There are other elements in an assembly language file; some of the most common are these:

- A comment contains text that will be ignored by the assembler. Comments begin with a ; (semicolon).
- A **symbol** is a text name that represents a value. For example, **Stack_Size** is a symbol.
- A **label** defines a symbol to refer to the current location in memory. For example, **Stack_Mem**, **__initial_sp**, **__Vectors** and **Reset_Handler** are all labels.

symbol

Text name representing a value (e.g. address, data value) in a program

label

Symbol in assembly language which represents an address

The object code created by the assembler is not complete if it has any external references. The assembler will include a list of external references and the instructions that use them. The linker/loader will resolve these references later.

```

;;;57      do {
000078  bf00                NOP
                        |L1.122|
;;;58      x += 2;
00007a  1c89            ADDS    r1,r1,#2
;;;59      } while (x < 20);
00007c  2914            CMP     r1,#0x14
00007e  d3fc            BCC     |L1.122|

```

Listing 5.2 Assembly language listing file created by C compiler (filename suffix is .txt).

There are tools to disassemble object code into a listing file that shows additional information, such as encoded instructions. The compiler may also create an assembly language listing file to show the code it has generated for its source file. Listing 5.2 shows an example. The file is in a similar format to the assembly language file, but includes additional information:

- A numerical address (typically hexadecimal) is listed for each memory location. The address may be relative (an offset from the beginning of the module) or absolute (a fixed location in memory). In Listing 5.2 the addresses are relative since linking (described in the next section) hasn't been performed yet.
- The content of a memory location, shown as a hexadecimal value. For example, relative address 0x00007E holds a value of 0xD3FC, which is the encoding of the instruction **BCC |L1.122|**. Contents that depend on external references will have placeholders because they are currently undefined.

Note that tools may use different names to refer to the same register. For example, r1 and R1 both refer to register one.

Linker/Loader

A linker/loader creates an executable file from multiple object files. These object files may come from modules in the source program or from libraries. The data and code objects are arranged in appropriate sections of memory. The linker can then determine the numerical addresses for variables and functions. These addresses are then used to complete the machine instructions that refer to the symbolic names. The resulting memory image is described in an executable file with the Arm ELF format and a filename suffix of .axf. Keil μ Vision uses the armlink linker [5].

Programmer

When the CPU is powered up or reset, it does not have an operating system to load a program into memory. The memory must already contain the program. The program memory is nonvolatile (typically Flash ROM), so it will retain its contents even after power is removed.

The programmer is a tool that places the program into the MCU's memory according to the description in the executable file. The programmer has both hardware and software. The hardware

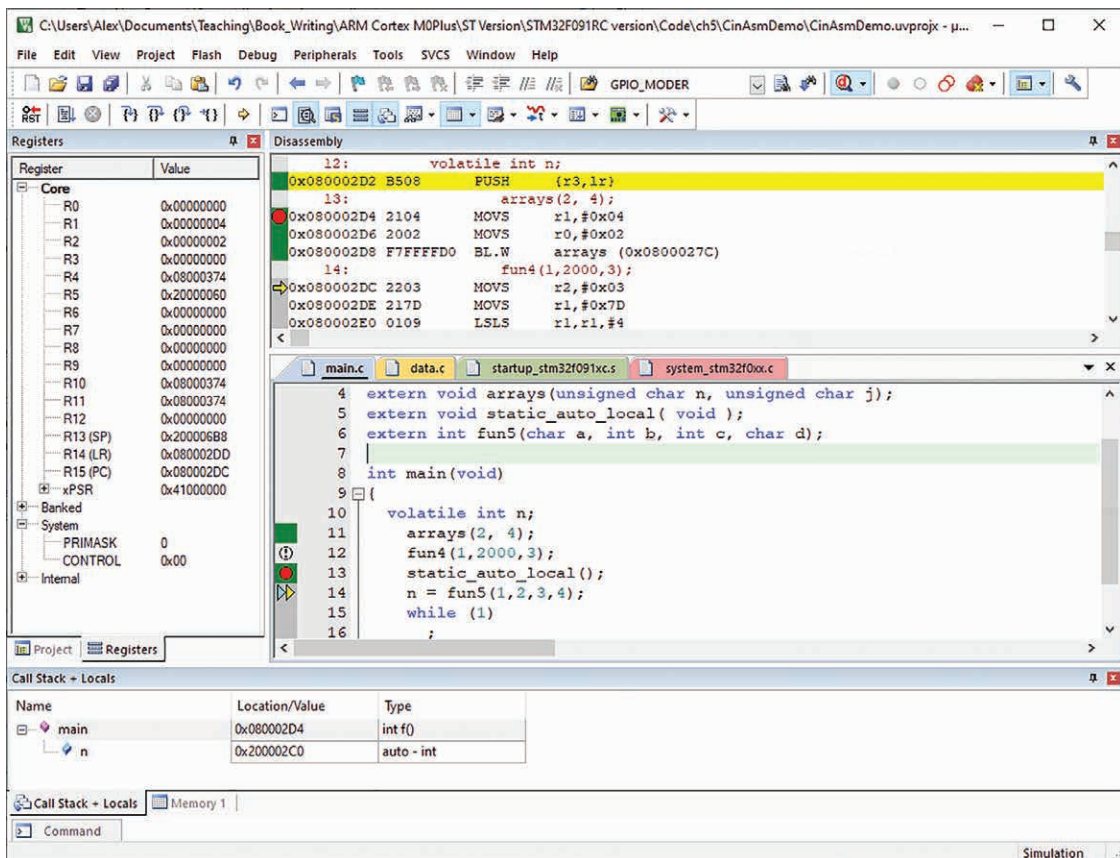


Figure 5.2 Debugger allows user to observe and control program execution, variables, and processor registers.

is connected to the MCU's serial wire debug (SWD) interface, enabling the MCU's memory to be programmed. The software may be a stand-alone program or it may be built into the IDE.

Debugger

The debugger enables the developer to control program execution and examine the program state (e.g. current instruction, values of processor registers, and data memory) as it runs on the processor. Figure 5.2 shows an example of the debugger's interface. The source code is shown in the central window (`delay.c`), while the corresponding object code is in the Disassembly window above. The Registers window to the left shows the values of the processor's core registers. The Call Stack + Locals window on the lower right shows both the current subroutine call nesting and the values of those functions' local variables.

C Language Fundamentals

Program and Functions

A program is made of one or more functions, with each made of a series of statements. A function may take arguments (also called parameters) and may return a result value. A function may call other functions sequentially or in a nested (hierarchical) way. The function *call graph* is a diagram that shows possible function calls. In Figure 5.3 the `main` function calls functions `J` and `K` as *subroutines*, and `J` calls `B` as a subroutine.

call graph

Diagram showing subroutine calling relationships between functions in a program

subroutine

Program function which can be called by another function

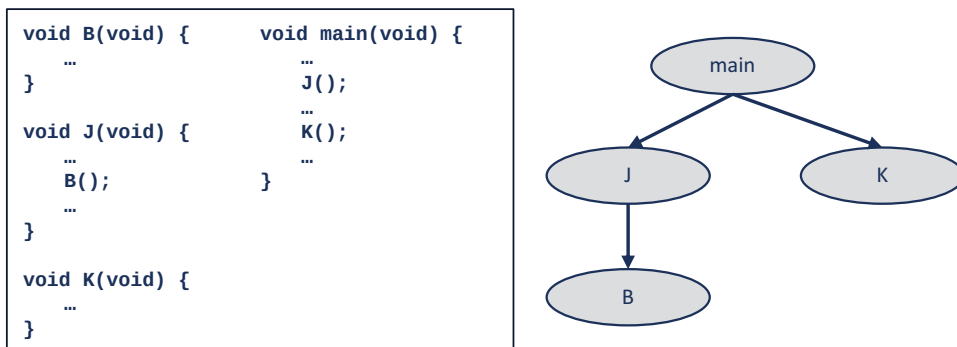


Figure 5.3 Source code and function call graph shows calling relationships between functions.

Every C program must have a function called **main**. Running the program consists of running the **main** function. The **main** function for embedded systems software never completes, unlike a program you might run on your personal computer or smart phone.

Functions use a **call stack** to hold temporary information. Calling a function creates a new **activation record** (stack frame) on top of the stack. Returning from a function destroys that activation record, freeing up the space for future function calls. For example, when **main** is executing, the call stack holds the activation record for **main**. When **J** is executing, the call stack holds the activation records for **J** and **main**. After **J** completes and control returns to **main**, the activation record for **J** is removed from the call stack.

call stack

Stack of activation records/stack frames of functions which have started executing but have not yet completed

activation record

Temporary storage in memory for function's preserved registers, arguments, local variables, return address, etc. Exists only from function's start to end.

Start-Up Code

When the CPU first starts running (e.g. after power-up or reset) it will execute the code for the reset exception handler. This is called **Reset_Handler** and is located in **startup_stm32f091xc.s**. The MCU is not ready for the user's **main** function yet and needs to be prepared.

First, some basic hardware settings may need to be configured. At startup the STM32F091RC MCU needs to configure a high-speed oscillator and select it after it has stabilized, initialize the vector table location and reset the PLL configurations. The reset exception handler performs these actions by calling the function **SystemInit** (defined in **system_stm32f0xx.c**).

Second, the runtime environment for the C program needs to be set up. For example, the stack pointer and variables need to be initialized. This is done in part by a runtime support function called **__scatterload**, which also sets certain variables to their correct initial values. We will discuss this further later in this chapter.

After performing these steps, the CPU can start executing the code in the **main** function.

Types of Memory

We saw in Chapter 4 that a microcontroller may use several types of memory. We can classify memory based on certain key characteristics:

- Can we write data to the memory? If so, how easy is it to do?
- Is the memory volatile or persistent? Volatile memory loses its contents when power is removed, whereas persistent memory does not.
- Does the memory need to be refreshed periodically?

Read-only memory (ROM) can only be read. It is nonvolatile (persistent) and retains its contents after power is removed. There are several types of ROM. The most basic ROM contains data that is specified when the IC is fabricated. Electrically erasable programmable ROM (EEPROM) can be erased and reprogrammed one location at a time. The erasing and programming operations take some time and may involve multiple steps. Flash EEPROM (typically called Flash or Flash ROM) allows an entire page of data to be erased or programmed at a time, saving time.

RAM is volatile and loses its contents when power is removed. There are two common types of RAM. Static RAM (SRAM) is built with digital latches, so it is fast and remembers data until power is removed. Dynamic RAM (DRAM) is built with a transistor acting like a capacitor, so it is slower and needs to be refreshed periodically. However, a DRAM cell is much smaller than an SRAM cell, so it is much less expensive.

MCUs typically have integrated Flash ROM and SRAM. Some may also have EEPROM to allow persistent storage of data that may need to change (e.g. configuration data). Some MCUs have (or can be configured to provide) address and data buses on their pins to allow external memory expansion. This allows external SRAM, DRAM, Flash ROM, and other devices to be added to the system. These MCUs typically also include DRAM memory refresh controllers.

A Program's Memory Requirements

What memory does a program need? Let's look at the several key characteristics of the information that will be stored in the memory.

- Does the information need to persist after power is removed? This information will need to be placed in ROM, EEPROM, or Flash ROM.
- Will the program only read the information? If so, this read-only (RO) information can be placed in ROM, Flash ROM, or EEPROM. If the program changes the information, this read/write (R/W) information will probably need to be placed in RAM.
- How long does the information need to exist? What is its **lifetime**? Information that must exist for the entire duration of the program must be given a permanent location in memory. This is called **static data**. Temporary information can be stored in read/write memory that is reused by different parts of the program. The program can use two reusable memory sections. The call stack is used to automatically allocate space on function entry and deallocate (free) it on function exit. The heap is used for explicit dynamic memory allocation and deallocation; the programmer must manage these operations.

Now we can look at the memory requirements of a C program. Figure 5.4 shows that data and instructions are allocated space in different memory areas.

Program instructions must be persistent and are generally read-only, so they are typically placed in Flash ROM. However, some MCUs also provide a small section of read/write memory to hold code that reprograms the Flash ROM when updating code.

Constant data values (**c**), data tables, text strings ("**He l l o !**"), and similar read-only items can be placed in Flash ROM.

Data variables can be read and written, so they must be placed in RAM. If a data variable is used only by a small part of the program, the compiler may optimize it by not allocating a memory location for it. Instead the compiler will just use a CPU register temporarily for the variable until the value is needed no more.

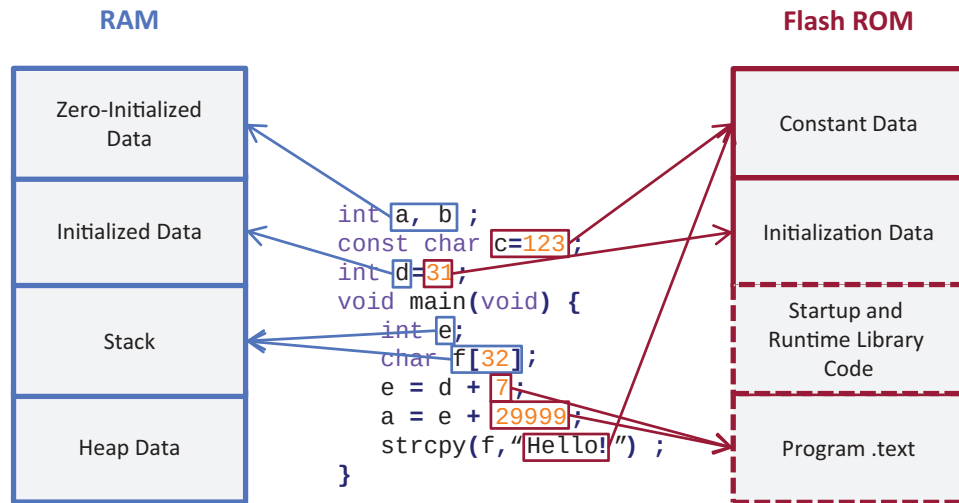


Figure 5.4 RAM and ROM are used to hold a program's data. Code is located in Flash ROM in the program, text or library regions.

Data variables are handled differently based on whether they are initialized and whether they are statically allocated. Later in this chapter we will see how variables in memory are accessed.

- Initialization is done in one of two ways. Static data (**d**) has a fixed memory address, so it is initialized at program start-up by copying the initial values from ROM. Automatically allocated data (**e, f**) is initialized by specific program instructions upon allocation, since the memory location may be used for different data before this use.
- Uninitialized variables are handled in two ways. Uninitialized static variables (**a, b**) have fixed memory addresses and they are initialized to zero. Uninitialized automatically or dynamically allocated variables (**e, f**) have undefined values, as their memory may have been used previously by other data.

Making Functions

A function's object code contains three sections. The **prolog** prepares the processor and memory, the **body** of the function performs the work specified by the source code's function body, and the **epilog** cleans up, prepares the return value (if any), and returns control to the calling function.

prolog

Initial code in function which preserves registers and prepares activation record

epilog

Final code in function which restores preserved registers, prepares return value, frees activation record and returns control to caller function

Register Use Conventions

The Arm architecture procedure calling standard (AAPCS) defines various behaviors, including how general-purpose registers are shared between a function and a subroutine. The calling function expects some registers to be overwritten by the subroutine, whereas others (called **preserved registers**) are expected to retain their values. If the subroutine needs to use a preserved register, it needs to save the value before overwriting it. Before completing, the subroutine needs to restore the preserved register to its original value.

Figure 5.5 shows that a subroutine must preserve the value of registers R5 through R11. Registers R0 through R3 can be modified without the need to be restored. Registers R0 through R3 may also be used for arguments, and R0 and R1 may be used for a return value.

Register Description	Symbol	On function exit, must be restored to original value?	Special use for subroutines?
Program Counter	PC (R15)	No	No
Link Register	LR (R14)	No	Holds return address after BL, BLX instructions
Stack Pointer	SP (R13)	Yes	Yes, points to top of stack
High general-purpose registers	R12	No	No
	R11	Yes	No
	R10	Yes	No
	R9	Yes	No
	R8	Yes	No
Low general-purpose registers	R7	Yes	No
	R6	Yes	No
	R5	Yes	No
	R4	Yes	No
	R3	No	Argument 4
	R2	No	Argument 3
	R1	No	Argument 2, result 2
	R0	No	Argument 1, result 1

Figure 5.5 Register use conventions.

Function Arguments

The arguments (or parameters) for a subroutine are passed according to the AAPCS. Arguments may be passed by register or by memory. Using registers is much faster, so the compiler tries to pass arguments in registers when possible. However, there are only four 32-bit registers available for argument use.

Each argument is extended to be a multiple of four bytes long. Arguments are then assigned to registers starting with R0; 64-bit arguments are allocated to even-numbered registers. Remaining arguments are passed on the stack.

For example, consider a function with three arguments: `char x`, `int y`, and `double z`. Argument `x` is eight bits long because it is of type `char`, so is extended to 32 bits and uses R0. Argument `y` is 32 bits long, so it uses R1 and does not need extension. Argument `z` is 64 bits long because it is a double precision float, so it uses R2 and R3. Now consider changing the order of the arguments to be `char x`, `double z`, and `int y`. Argument `x` is passed in R0, argument `z` is passed in R2 and R3, and argument `y` is passed on the stack.

Function Return Value

A function returns a value using R0, R1, or the stack. Return value types up to 32 bits use R0; shorter types are extended to 32 bits as mentioned earlier. Return value types up to 64 bits use R0 and R1, with shorter types extended. Longer types are returned on the stack.

Prolog and Epilog

In this section we will examine the prolog and epilog for the C function shown in Listing 5.3.

```
int fun4(char a, int b, char c) {
    int x[8];
    x[0] = a * b;
    x[c] = b;
    return a+b+c;
}
```

Listing 5.3 Example source code.

Prolog

The prolog has several responsibilities, including saving preserved registers, setting up the activation record on the call stack, and initializing automatic variables when needed.

Registers R4 through R11 must be preserved across function calls, as specified in Figure 5.5. If the body of the function might use any of these registers, then the prolog will save these registers on the stack with a **PUSH** instruction. If the body of the function might make a subroutine call, then the return address in LR must also be saved. This is the address of the next instruction to execute in the calling function after the subroutine completes. In Listing 5.4, the **PUSH** instruction will save R4 and the link register on the stack.

```
                fun4 PROC
;;;101    int fun4(char a, int b, char c) {
0000ba    b510                PUSH    {r4, lr}
0000bc    b088                SUB     sp, sp, #0x20
;;;102    int x[8];
```

Listing 5.4 Prolog code for function `fun4` saves register R4 and link register, then allocates 32 bytes of space on call stack for integer array `x`.

The prolog may allocate space on the stack for automatic variables. In Listing 5.4, the **SUB** instruction will grow the stack by subtracting 0x20 (32 decimal) from the stack pointer. This allocates the space needed for the automatic variable `x`, an array of eight integers. As each integer takes 4 bytes, 32 bytes are needed in total. Note that the compiler tries to promote variables from stack memory into registers, so some automatic variables will use only registers and no stack space.

	Memory Address	Contents	Description
3. SP after SUB sp,sp,#0x20 →	A – 0x28	x[0]	Array x
	A – 0x24	x[1]	
	A – 0x20	x[2]	
	A – 0x1C	x[3]	
	A – 0x18	x[4]	
	A – 0x14	x[5]	
	A – 0x10	x[6]	
	A – 0x0C	x[7]	
2. SP after PUSH {r4,lr} →	A – 0x08	r4	Preserved register
	A – 0x04	LR	Return address
1. SP on entry to function, before PUSH {r4,lr} →	A		Caller's stack frame

Figure 5.6 Activation record creation.

Figure 5.6 shows how the **PUSH** and **SUB** instructions create the activation record on the stack.

Epilog

The epilog needs to place the return value (if present) in the correct location, restore preserved registers to their original values, and return control to the calling function.

When a subroutine is called, the return address is placed in the link register (LR) by the branch and link (**BL**) or branch and link and exchange (**BLX**) instruction. Control can be returned to the calling function by executing the **BX LR** instruction, resulting in a branch to the address stored in LR. However, if this subroutine has called another subroutine, then the first return address (in LR) will be overwritten by the second call. To prevent this, a subroutine that may call another subroutine will save the return address on the stack with a **PUSH {LR}** instruction. In this case, the return instruction will be **POP {PC}**, which pops the return address from the stack into the program counter. Note that if other registers were pushed onto the stack in the prolog, the **POP** instruction may have a list of multiple registers.

```

;;;105      return a+b+c;
0000ca 1840      ADDS    r0,r0,r1
0000cc 1880      ADDS    r0,r0,r2
;;;106      }
0000ce b008      ADD     sp,sp,#0x20
0000d0 bd10      POP     {r4,pc}
ENDP

```

Listing 5.5 Epilog code for function **fun4** computes return value, deallocates 32 bytes from stack, then restores original values of registers R4 and link register.

Listing 5.5 shows that the source code return statement is implemented with two **ADDS** instructions that place the result (**a+b+c**) in **R0**. The next **ADD** instruction adds **0x20** (32 decimal) to the stack pointer to deallocate the space for the array **x**. The last instruction (**POP**) has two effects. First, it restores **R4** to its original saved value, as required by the register use conventions. Second, it loads the **PC** with the saved value of the link register, which is the address of the instruction in the calling function that follows the subroutine call. After the **POP** instruction executes, the CPU will continue executing the calling function.

Figure 5.7 shows how the **ADD** and **POP** instructions delete the activation record from the stack.

	Memory Address	Contents	Description
1. SP before <i>ADD sp,sp,#0x20</i> →	A – 0x28	x[0]	Array x
	A – 0x24	x[1]	
	A – 0x20	x[2]	
	A – 0x1C	x[3]	
	A – 0x18	x[4]	
	A – 0x14	x[5]	
	A – 0x10	x[6]	
	A – 0x0C	x[7]	
2. SP after <i>ADD sp,sp,#0x20</i> →	A – 0x08	r4	Preserved register
	A – 0x04	LR	Return address
3. SP after <i>POP {r4,pc}</i> →	A		Caller's stack frame

Figure 5.7 Activation record deletion.

Exception Handlers

The compiler generates code for exception handlers (including interrupt service routines) in a similar way to regular functions. The compiler identifies an exception handler with the **__irq** qualifier in its declaration. There are three main differences between exception handlers and regular functions.

First, a handler must have no arguments or return values. The compiler will signal an error if a function declared with **__irq** takes arguments or returns a value. Recall that the CMSIS support for an MCU declares standard names for the MCU's exception handlers; these declarations include the **__irq** qualifier.

Second, because the handler can execute anywhere in the program, **all registers** must be treated as preserved registers. When responding to an exception, the CPU automatically saves certain general-purpose registers on the stack: **R0**, **R1**, **R2**, **R3**, and **R12**. The handler does not need to save the value of these registers. If the handler uses any other general-purpose registers (**R4** through **R11**), it will need to save their values upon entry and restore them before exiting. The C compiler generates prolog and epilog code for a handler that does this.

Third, the handler must return using an instruction that triggers exception return processing.

There is no “return from interrupt” instruction for ARMv6-M processors. Instead, we use an instruction to update the PC with a special exception return code (EXC_RETURN) to trigger the CPU’s exception processing hardware and specify which stack pointer and processor mode to use. Recall that this code was loaded into the link register when first responding to an exception. If the exception return code is still in the link register, we use a branch indirect to the link register (BX LR). Otherwise the code was pushed onto the stack because the handler may have called a subroutine, so we need to pop the code from the stack into the PC.

```

                                TIM6_DAC_IRQHandler PROC
;;;15
;;;16    void TIM6_DAC_IRQHandler(void) {
000000    b430                PUSH    {r4,r5}
; (handler body code here)
000086    bc30                POP     {r4,r5}
000088    4770                BX      lr
;;;61
                                ENDP

```

Listing 5.6 Prolog and epilog of exception handler.

Listing 5.6 shows the prolog and epilog of a timer peripheral’s ISR in assembly code. Note that registers R4 and R5 are preserved with a **PUSH** instruction. The body of the handler uses registers R0 through R5. The values of R4 and R5 are restored with the **POP**. The return from exception operation is performed by executing the **BX lr** instruction; this restores the values of R0 through R3 (as well as R12, LR, PC, and xPSR).

Controlling the Program’s Flow

In this section we examine the assembly code that implements the C control flow structures for conditionals and loops and subroutine calls.

Conditionals

The C language offers if/else and switch code structures to select one of the multiple code blocks to execute.

If/Else

```

if (x){
    y++;
} else {
    y--;
}

```

Listing 5.7 C source code with if/else statement.

If/else statements are simple, requiring a test, code for the true (**if**) case and code for the false (**else**) case. In Listing 5.7 the if statement tests the value of variable **x**. If **x** is nonzero, then the **y++** statement is executed. If **x** is zero, then the **y--** statement is executed. Listing 5.8 shows the assembly code generated by the compiler.

```

;;;39      if (x){
000056  2900          CMP    r1,#0
000058  d001          BEQ    |L1.94|
;;;40      y++;
00005a  1c52          ADDS    r2,r2,#1
00005c  e000          B        |L1.96|
                        |L1.94|
;;;41      } else {
;;;42      y--;
00005e  1e52          SUBS    r2,r2,#1
                        |L1.96|
;;;43      }

```

Listing 5.8 Assembly code listing for if/else statement.

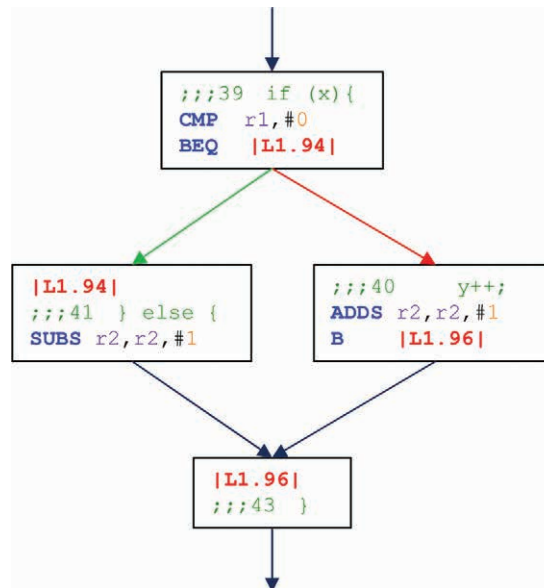


Figure 5.8 Control flow of if/else statement code.

Figure 5.8 shows the control flow of the assembly code. Register **R1** holds the variable **x**.

- The **CMP r1, #0** instruction (at address 0x000056) compares **R1** with the immediate value zero and sets the processor's condition code flags according to the result. If the two values (**R1** and zero) are equal, then the **Z** flag will be set to one. Otherwise it will be cleared to zero.
- The **BEQ |L1.94|** instruction (at address 0x000058) will branch to label **|L1.94|** if the values are equal (since the **Z** flag is set). Otherwise the **BEQ** instruction will allow program execution to continue to the next instruction (at address 0x00005A).

- The code for the true case starts at address 0x00005A. The `ADDS r2, r2, #1` instruction adds one to R2, which is used for the variable `y`. The `B |L1.96|` instruction (at address 0x00005C) forces the program to branch to label `|L1.96|` to skip over the false case code.
- The code for the false case starts at address 0x00005E, which is also the value of label `|L1.94|`. The instruction `SUBS r2, r2, #1` subtracts one from register R2, which is used for the variable `y`.
- The label `|L1.96|` marks the merge point of the if and else cases, and is the address of the first instruction after the if/else statement.

Switch

```
switch (x) {
case 1:
    y += 3;
    break;
case 31:
    y -= 5;
    break;
default:
    y--;
    break;
}
```

Listing 5.9 C source code with switch statement.

Switch statements can be implemented in different ways. The approach shown in Listing 5.10 performs a test for each case, similar to the if/else structure. Other approaches are to use a jump table or a computed jump, which eliminate the need for multiple tests. Figure 5.9 shows the control flow of the assembly code.

```
;;;45      switch (x) {
000060  2901          CMP        r1, #1
000062  d002          BEQ        |L1.106|
000064  291f          CMP        r1, #0x1f
000066  d104          BNE        |L1.114|
000068  e001          B          |L1.110|
                                |L1.106|
;;;46      case 1:
;;;47          y += 3;
00006a  1cd2          ADDS       r2, r2, #3
;;;48          break;
00006c  e003          B          |L1.118|
                                |L1.110|
;;;49      case 31:
;;;50          y -= 5;
00006e  1f52          SUBS       r2, r2, #5
;;;51          break;
000070  e001          B          |L1.118|
                                |L1.114|
```

```

;;;52      default:
;;;53      y--;
000072 1e52      SUBS    r2,r2,#1
;;;54      break;
000074 bf00      NOP
                |L1.118|
000076 bf00      NOP                                ;48
;;;55      }

```

Listing 5.10 Assembly code listing of switch statement.

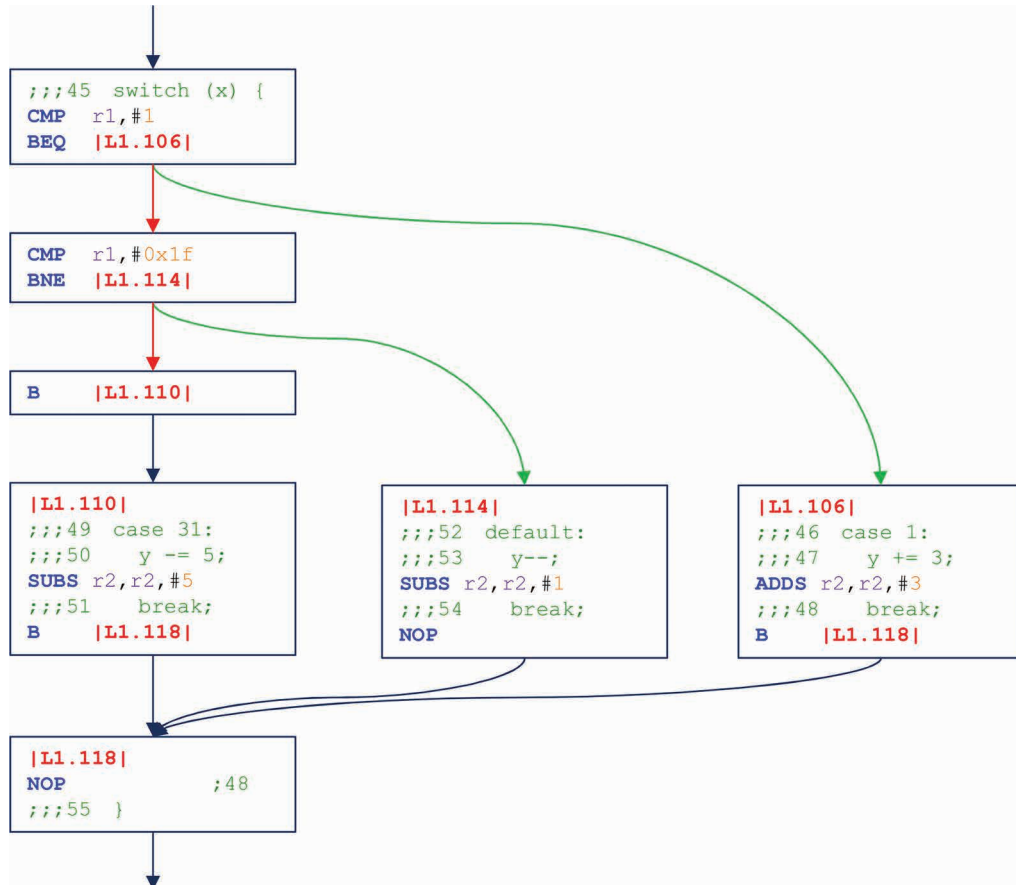


Figure 5.9 Control flow of switch statement code.

There are three different cases based on the value of *x*: a case for 1, a case for 31, and a default case. The code starts by comparing *x* to each of the values.

- The instruction `CMP r1, #1` (at address 0x000060) compares *x* to 1.
- If they are equal, then the `BEQ |L1.106|` instruction (at address 0x000062) causes the program to branch to the corresponding case code at label `|L1.106|` (at address 0x00006A).

- If R1 is not equal to 1, then the code will continue with the next test instruction: **CMP r1, #0x1f** (at address 0x00006E), and #0x1f is the immediate value of 31 represented in hexadecimal.
- If they are not equal, then the **BNE |L1.114|** instruction (at address 0x000066) causes the program to branch to the default case code at label |L1.114| (at address 0x000072).
- If R1 is equal to 31, then the code will continue with the next instruction, which is **B |L1.110|** (at address 0x000068). This will cause the program to jump to label |L1.110| (at address 0x00006E) and execute the code starting there.
- The code for each of the cases is similar. First, register R2 (holding the variable y) is modified. Then the program unconditionally branches to the merge point of |L1.118| (at address 0x000076) because of the break statement. The last case does not need such a branch case because the execution will naturally proceed to the merge point. A **NOP** instruction is added as a placeholder for debugging at address 0x000074.

Loops

Code structures for loops include a loop body and a loop test. The test may be performed before the body is executed (top-test) or after (bottom-test).

Do While

```
do {
    x += 2;
} while (x < 20);
```

Listing 5.11 C source code with do/while loop.

The do/while loop is simple, executing the loop body first (adding 2 to x) and then testing whether to repeat the body (if x < 20). The assembly code is shown in Listing 5.12, whereas Figure 5.10 shows the control flow.

```
;;;57      do {
000078  bf00      NOP
                        |L1.122|
;;;58      x += 2;
00007a  1c89      ADDS      r1, r1, #2
;;;59      } while (x < 20);
00007c  2914      CMP        r1, #0x14
00007e  d3fc      BCC        |L1.122|
```

Listing 5.12 Assembly code of do/while loop.

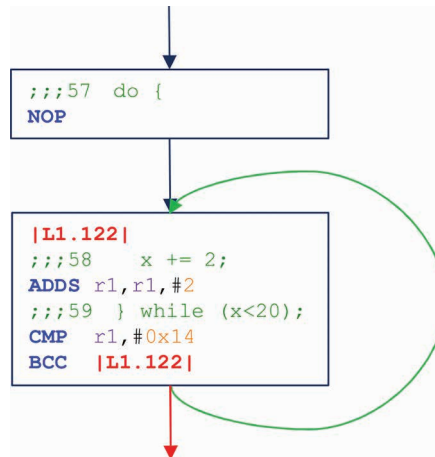


Figure 5.10 Control flow of do/while loop.

- A **NOP** instruction (address 0x000078) serves as a placeholder for debugging.
- The body of the loop adds 2 to *x* using the instruction **ADDS r1, r1, #2** (address 0x00007A).
- The loop test starts by comparing *x* to 20 (hexadecimal 0x14) using **CMP r1, #0x14** (address 0x00007C). This instruction checks for the result of subtracting R1 from 20, but doesn't update R1. Instead, it only updates the condition code flags.
- The loop test then can branch back to the loop body or else continue with the next instruction. If *x* is less than 20, then the comparison mentioned does not result in a borrow (indicated by the carry flag), so the C flag will be zero. Branch if carry cleared (**BCC**) performs a branch if the carry bit is cleared (zero). If *x* is not less than 20, then the C flag will be one, so the **BCC** will not execute, and the program will proceed to the next instruction after the loop.

While

```
while (x<10) {
    x = x + 1;
}
```

Listing 5.13 C source code with while loop.

The while loop's assembly code appears in Listing 5.14, and Figure 5.11 shows the control flow. The while loop performs the test first, and then executes the loop body. However, the code is laid out in a different order. The start of the loop has a branch over the loop body to the test at the end. So the loop test is executed first, before the body has a chance to execute.

```

;;;61      while (x<10) {
000080  e000          B          |L1.132|
          |L1.130|
;;;62      x = x + 1;
000082  1c49      ADDS      r1,r1,#1
          |L1.132|
000084  290a      CMP      r1,#0xa          ;61
000086  d3fc      BCC      |L1.130|

;;;63      }

```

Listing 5.14 Assembly code listing of while loop.

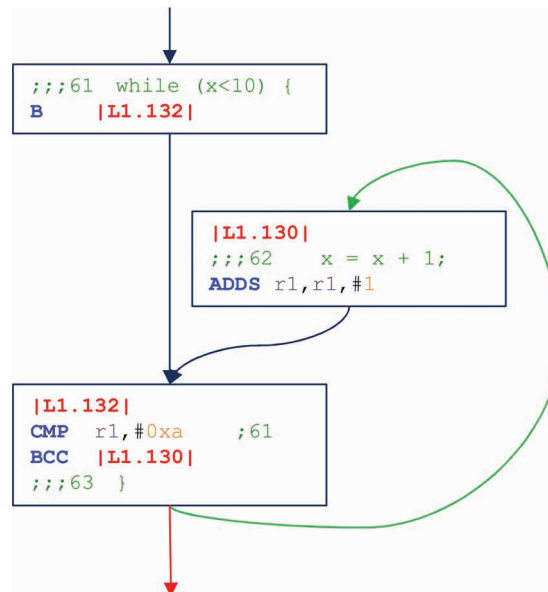


Figure 5.11 Control flow of while loop.

- **B |L1.132|** (at address 0x000080) branches to label **|L1.132|**, which is the start of the loop test.
- The loop test starts at **|L1.132|**. Instruction **CMP r1, #0xa** (address 0x000084) compares R1 to 10 (0xa in hexadecimal) and sets the condition code flags according to the result.
- If R1 is less than 10, then the Carry flag will be cleared, so **BCC |L1.130|** (address 0x000086) will cause the program to branch to that label.
- The loop body starts at label **|L1.130|** with the instruction **ADDS r1, r1, #1** (address 0x000082) that adds 1 to register R1 (holding variable x).

For

```
for (i = 0; i < 10; i++){
    x += i;
}
```

Listing 5.15 C source code with for loop.

The for loop is the most complex loop. It contains initialization code (`i = 0`) that executes before the loop starts, a loop test (`i < 10`), the loop body (`x += i`), and loop index update code (`i++`). The assembly code appears in Listing 5.16, while Figure 5.12 shows the control flow.

```
;;;65      for (i = 0; i < 10; i++){
000088 2300          MOVS    r3,#0
00008a e001          B      |L1.144|
                |L1.140|
;;;66      x += i;
00008c 18c9          ADDS    r1,r1,r3
00008e 1c5b          ADDS    r3,r3,#1          ;65
                |L1.144|
000090 2b0a          CMP     r3,#0xa          ;65
000092 d3fb          BCC    |L1.140|

;;;67      }
```

Listing 5.16 Assembly code listing of for loop.

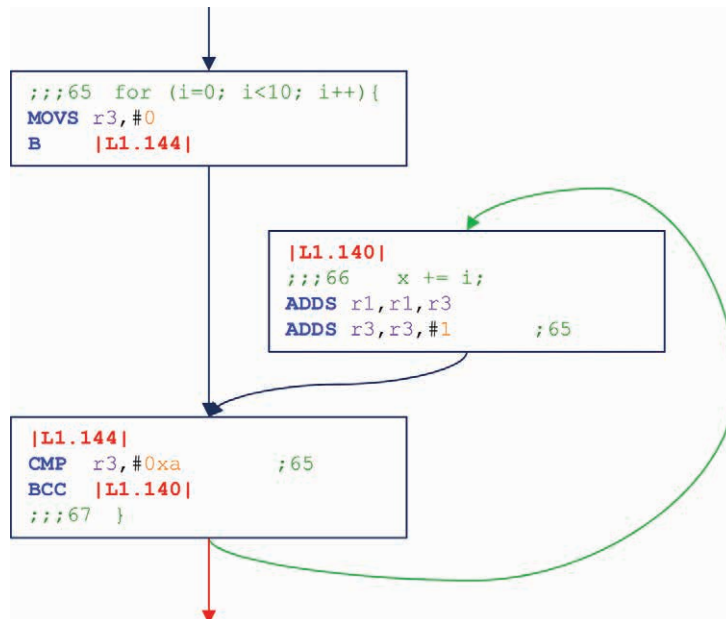


Figure 5.12 Control flow of for loop.

- The loop initialization code (`i = 0`) is performed by `MOVS r3, #0` (address 0x000088). Register R3 holds variable `i`, which serves as a loop counter. The code then branches to the loop test with the instruction `B |L1.144|` (address 0x00008A).
- The loop test code is marked with label `|L1.144|` and starts with instruction `CMP r3, #0xa` (address 0x000090). This compares the loop counter `i` (R3) with 10 (hexadecimal 0xA). The branch instruction `BCC |L1.140|` will branch to the loop body if R3 is less than 10.
- The loop body code is marked with label `|L1.140|`. It uses the instruction `ADDS r1, r1, r3` (address 0x00008C) to add `i` to `x`.
- The loop index update code follows the loop body. It uses the instruction `ADDS r3, r3, #1` (address 0x00008E) to add one to the variable `i`.

Note that the code is laid out in the sequence of initialization code, loop body, loop index update, and test. However, it is executed in a different order, with the test performed before the loop body.

Calling Subroutines

```
extern int fun5(char a, int b, int c, char d);
int main(void)
{
    int n;
    n = fun5(1,2,3,4);
}
```

Listing 5.17 C source code calling function `fun5` as subroutine.

Calling a subroutine requires preparing the arguments (if any) and then calling the subroutine. After the return, the result value (if any) can be used. Let's consider the C source code shown in Listing 5.17, which shows `main` calling `fun5` as a subroutine. The function `fun5` has four arguments: one char (`a`), two ints (`b`, `c`), and another char (`d`). The assembly code is presented in Listing 5.18.

```
;;;16          n = fun5(1,2,3,4);
00001a 2304          MOVS    r3, #4
00001c 2203          MOVS    r2, #3
00001e 2102          MOVS    r1, #2
000020 2001          MOVS    r0, #1
000022 f7ffffffe       BL      fun5
000026 9000          STR     r0, [sp, #0]
```

Listing 5.18 Assembly code calling function `fun5` as subroutine.

First the arguments are loaded into the appropriate registers: R0 for argument `a`, R1 for `b`, R2 for `c`, and R3 for `d`. The arguments are loaded in reverse order, but any order would work.

Second, the subroutine is called either with `BL` or `BLX` instruction. The `BL` instruction contains the address of the subroutine. The `BLX` instruction needs the subroutine's address to be loaded into register R0. Both of these instructions will place the address of the following instruction in the LR to allow the subroutine to return control to the calling function. Listing 5.18 shows that the subroutine is called with the instruction `BL fun5` (at address 0x000022).

Third, after `fun5` completes its returns control to `main`, its result will be in `R0`. The instruction `STR r0, [sp, #0]` (at address `0x000026`) stores that result to the memory for the `main`'s automatic variable `n`.

Accessing Data in Memory

Let's see how to access data in memory. A data variable must be in a register in order for the program to operate on it. The access method depends on the variable's location (e.g. static memory vs. on the stack) and other factors (e.g. using pointers, whether in an array).

Statically Allocated Memory

A variable in a statically allocated memory can be anywhere in the CPU's 32-bit memory space, so we need 32 bits to specify its address. There is not enough space to hold both a 32-bit address and an operation specifier in a Thumb instruction, since most are 16 bits long (with a few being 32 bits long).

To solve this problem, we take advantage of the program-counter-relative addressing mode. The variable's address is stored in memory near the instructions that need it. The program uses an `LDR` instruction to load a register with the variable's address. The program then uses that register to specify the memory location.

```

;;;20      siA = 2;
00000e 2102      MOVS    r1, #2
000010 4a49      LDR     r2, |L1.312|
000012 6011      STR     r1, [r2, #0] ; siA
;;;21      aiB = siC + siA;
000014 4949      LDR     r1, |L1.316|
000016 6809      LDR     r1, [r1, #0] ; siC
000018 6812      LDR     r2, [r2, #0] ; siA
00001a 1889      ADDS    r1, r1, r2
00001c 9103      STR     r1, [sp, #0xc]

; Pointers to static data
|L1.312|
DCD     ||siA||
|L1.316|
DCD     ||siC||

; Static data
AREA    ||.data||, DATA, ALIGN=2
||siA||
DCD     0x00000000
||siC||
DCD     0x00000003

```

Listing 5.19 Code to access variables in statically allocated memory.

For example, the code in Listing 5.19 first assigns a value of 2 to static integer `siA`, and then reads the values of `siC` and `siA`. The variables `siA` and `siC` are allocated space in the data memory section with the `DCD` assembler directives.

Automatically Allocated Memory

Automatically allocated variables are stored on the stack. Each such variable is located in memory at a specific offset from the stack pointer. This `sp`-relative addressing mode is specified in assembly code as `[sp, #offset]`. The offset ranges from 0 to 1020 and must be a multiple of four.

```

;;;14  void static_auto_local( void ) {
000000 b50f      PUSH      {r0-r3, lr}
;;;15      int aiB;
;;;16      static int siC=3;
;;;17      int * apD;
;;;18      int aiE=4, aiF=5, aiG=6;
000002 2104      MOVS      r1, #4
000004 9102      STR       r1, [sp, #8]
000006 2105      MOVS      r1, #5
000008 9101      STR       r1, [sp, #4]
00000a 2106      MOVS      r1, #6
00000c 9100      STR       r1, [sp, #0]

```

Listing 5.20 Code to access variables in automatically allocated memory.

The three store instructions in Listing 5.20 initialize the variables `aiE`, `aiF`, and `aiG` in memory at specific offsets from the stack pointer. The activation record on the stack is shown in Figure 5.13.

Address	Contents	Description
SP	aiG	Automatic variables
SP + 0x04	aiF	
SP + 0x08	aiE	
SP + 0x0c	aiB	
SP + 0x10	r0	Preserved registers
SP + 0x14	r1	
SP + 0x18	r2	
SP + 0x1c	r3	
SP + 0x20	LR	Return address
SP + 0x24		Caller's stack frame

Figure 5.13 Contents of activation record.

Dynamically Allocated Memory and Other Pointers

A pointer variable holds the address of a data item, such as another data variable. Pointers are used to access memory that is dynamically allocated (using `malloc`, `calloc`, or `realloc`).

```

;;;22      apD = & aiB;
00001e  a803      ADD      r0, sp, #0xc
;;;23      (*apD)++;
000020  6801      LDR      r1, [r0, #0]
000022  1c49      ADDS     r1, r1, #1
000024  6001      STR      r1, [r0, #0]

```

Listing 5.21 Code to access a variable using a pointer.

The statement `apD = & aiB` loads the address of the variable `aiB` into the variable `apD`. Listing 5.21 shows how this is implemented in assembly code. The variable `aiB` is located at the location `SP+0x0c`, as shown in Figure 5.13. The instruction at address `0x00001E` calculates this address by adding `SP` and `0x0C` and then places the result in `R0`, which holds `apD` and serves as a pointer to `aiB`.

The statement `(*apD)++` will increment the variable pointed to by `apD`. This is performed in three steps:

- The `LDR` instruction (address `0x000020`) loads `R1` with the memory value to which `apD` points (which is `aiB`),
- The `ADDS` instruction (address `0x000022`) adds one to `R1`
- The `STR` instruction (address `0x000024`) stores `R1` back to memory via pointer `apD`.

Array Elements

In order to access an array element, its address must be determined, which is the sum of the array's starting address and an offset. For a one-dimensional array, the offset is the product of the element size (in bytes) and index of the particular element. For a two-dimensional array, the offset must also include the size of rows before the desired element's row.

Let's look at the one-dimensional array declared as `unsigned char buff2[3]`. This array of characters has three elements. Each element is a character, so each takes one byte. The entire array takes three bytes and is laid out in memory as shown in Figure 5.14.

Address	Contents
<code>buff2</code>	<code>buff2[0]</code>
<code>buff2 + 1</code>	<code>buff2[1]</code>
<code>buff2 + 2</code>	<code>buff2[2]</code>

Figure 5.14 Memory layout of array declared as `unsigned char buff2[3]`.

```

;;;72  unsigned int arrays(unsigned char n, unsigned char j) {
00009c b508          PUSH    {r3,lr}
00009e 4602          MOV     r2,r0
;;;73          volatile unsigned int i;
;;;74
;;;75          i = buff2[0] + buff2[n];
0000a0 4827          LDR     r0, |L1.320|
0000a2 7800          LDRB    r0, [r0,#0] ; buff2
0000a4 4b26          LDR     r3, |L1.320|
0000a6 5c9b          LDRB    r3, [r3,r2]
0000a8 18c0          ADDS    r0,r0,r3
0000aa 9000          STR     r0, [sp,#0]
; Static data
          |L1.320|
          DCD     buff2

```

Listing 5.22 Code to access and add two elements in a one-dimensional array.

The code to add two elements in the array (`buff2[0]` and `buff2[n]`) is shown in Listing 5.22 and explained in Figure 5.15.

Instruction	Description
00009c PUSH {r3,lr}	This instruction saves R3 and the return address on the stack.
00009e MOV r2,r0	The parameter n is passed into the function through register 0. This instruction copies that value into register R2, freeing up r0 for other use.
0000a0 LDR r0, L1.320	The starting address of the array buff2 is loaded into register R0.
0000a2 LDRB r0, [r0,#0]	The code needs to calculate the offset of each element from the array's starting address (buff2). Because the first element is a constant (zero), the offset will be a constant which the compiler can calculate. This simplifies the assembly code. Element 0 is located at an offset of 1 byte/element * 0 elements = 0 bytes. So the address of buff2[0] is buff2 + 0, or simply buff 2. This instruction reads a byte from memory at location R0 and places the result in R0.
0000a4 LDR r3, L1.320	The starting address of the array buff2 is loaded into register R3 with this instruction.
0000a6 LDRB r3, [r3,r2]	The offset of buff2[n] is 1 byte/element * n elements = n bytes. The address of buff2[n] is therefore buff2 + n. This instruction reads a byte from memory at location R3+R2 and places the result in register R3.
0000a8 ADDS r0,r0,r3	This instruction adds R0 (element buff2[0]) and R3 (element buff2[n]) and places the result in R0.
0000aa STR r0, [sp,#0]	This instruction stores the sum on the stack at offset 0.

Figure 5.15 Explanation of code implementing `i = buff2[0] + buff2[n]`.

Now let's look at the two-dimensional array declared as `short int buff3[5][7]`. This array of short integers has five rows and seven columns, with seven elements per column. Each element is a short integer, so each takes two bytes. The entire array takes $2 \times 5 \times 7 = 70$ bytes and is laid out in memory as shown in Figure 5.16.

The code to add the element `buff3[n][j]` to variable `i` is shown in Listing 5.23. The element's address is calculated based on several parts: the array's starting address, the row offset (based on the row size and the row number), and the column offset (based on the column number and the element size). Each instruction in the code is explained in Figure 5.17.

Address	Contents	Comment
buff3	buff3[0][0]	Row 0
buff3 + 1		
buff3 + 2	buff3[0][1]	
buff3 + 3		
(etc.)		
buff3 + 10	buff3[0][5]	
buff3 + 11		
buff3 + 12	buff3[0][6]	Row 1
buff3 + 13		
buff3 + 14	buff3[1][0]	
buff3 + 15		
buff3 + 16	buff3[1][1]	
buff3 + 17		
buff3 + 18	buff3[1][2]	
buff3 + 19		
(etc.)		
buff3 + 68	buff3[4][6]	Row 4
buff3 + 69		

Figure 5.16 Memory layout of array declared as `short int buff3[5][7]`.

```

;;;76      i += buff3[n][j];
0000ac 200e      MOVS      r0, #0xe
0000ae 4350      MULS      r0, r2, r0
0000b0 4b24      LDR       r3, |L1.324|
0000b2 18c0      ADDS      r0, r0, r3
0000b4 004b      LSLS      r3, r1, #1
0000b6 5ac0      LDRH      r0, [r0, r3]
0000b8 9b00      LDR       r3, [sp, #0]
0000ba 18c0      ADDS      r0, r0, r3
0000bc 9000      STR       r0, [sp, #0]
; Static data
                                |L1.324|
                                DCD      buff3

```

Listing 5.23 Code to access element in two-dimensional array.

Instruction	Description
0000ac MOVS r0, #0xe	The row size is two bytes/element * 7 elements per row = 14 bytes. This instruction loads the hexadecimal value 0xe (which is decimal 14) into r0.
0000ae MULS r0, r2, r0	The row offset is the row size multiplied by the row number (n, which is still in r2). This instruction calculates the row offset.
0000b0 LDR r3, L1.324	The starting address of the array buff3 is loaded into register r3 with this instruction.
0000b2 ADDS r0, r0, r3	The starting address (r3) and the row offset (r0) are added with this instruction and placed back in r0.
0000b4 LSLS r3, r1, #1	The column offset is element's column number multiplied by the number of bytes per element (two). The column number j is passed as an argument through r1. It is multiplied by two by shifting it left by one bit position with this instruction and stored in r3.
0000b6 LDRH r0, [r0, r3]	The array element's address is the sum of the base address and the row offset (in r0) and the column offset (in r3), and is formed with [r0, r3]. The halfword at that address is loaded into register r0.
0000b8 LDR r3, [sp, #0]	Register r3 is loaded with the value of variable i, which is located on the stack at offset 0.
0000ba ADDS r0, r0, r3	The array element and i are added together and placed in r0.
0000bc STR r0, [sp, #0]	The sum calculated above is stored to the memory location for variable i.

Figure 5.17 Explanation of code implementing `i += buff3[n][j]`.

Summary

In this chapter, we have examined the assembly code that the compiler generates to implement the C language source program. We examined the program build tools, which translate program modules between languages and then link them together. We then saw how functions are built from a prolog, an epilog, and a body with control flow and data access operations. We also evaluated how exception handlers differ from regular functions.

Exercises

Consider the following assembly code that the compiler has generated for a C function. Explain what each assembly instruction does and describe what data is in any registers used.

	Assembly Code Listing		Explanation
1.	;;;5	void fn(int8_t * a, int32_t * b,	
	float * c) {		
	000000	b5f0	PUSH {r4-r7,lr}
2.	000002	b085	SUB sp,sp,#0x14
3.	000004	4604	MOV r4,r0
4.	000006	460d	MOV r5,r1
5.	000008	4616	MOV r6,r2
6.	;;;6	volatile int8_t a1, a2;	
	;;;7	volatile int32_t b1, b2;	
	;;;8	volatile float c1, c2;	
	;;;9		
	;;;10	a1 = 15;	
	00000a	270f	MOVS r7,#0xf
7.	;;;11	a2 = -14;	
	00000c	200d	MOVS r0,#0xd
8.	00000e	43c0	MVNS r0,r0
9.	000010	9004	STR r0,[sp,#0x10]
10.	;;;12	*a = a1*a2;	
	000012	9804	LDR r0,[sp,#0x10]
11.	000014	4378	MULS r0,r7,r0
12.	000016	b240	SXTB r0,r0
13.	000018	7020	STRB r0,[r4,#0]
14.	;;;13		
	;;;14	b1 = 15;	
	00001a	200f	MOVS r0,#0xf
15.	00001c	9003	STR r0,[sp,#0xc]
16.	;;;15	b2 = -14;	
	00001e	200d	MOVS r0,#0xd
17.	000020	43c0	MVNS r0,r0
18.	000022	9002	STR r0,[sp,#8]
19.	;;;16	*b = b1*b2;	
	000024	9902	LDR r1,[sp,#8]
20.	000026	9803	LDR r0,[sp,#0xc]
21.	000028	4348	MULS r0,r1,r0
22.	00002a	6028	STR r0,[r5,#0]
23.	;;;17		
	;;;18	c1 = 15;	
	00002c	4809	LDR r0, L1.84
24.	00002e	9001	STR r0,[sp,#4]

	Assembly Code Listing			Explanation
25.	;;;19	c2 = -14;		
	000030	4809	LDR	r0, L1.88
26.	000032	9000	STR	r0, [sp, #0]
27.	;;;20	*c = c1*c2;		
	000034	9900	LDR	r1, [sp, #0]
28.	000036	9801	LDR	r0, [sp, #4]
29.	000038	f7ffffffe	BL	__aeabi_fmul
30.	00003c	6030	STR	r0, [r6, #0]
31.	;;;21			
	;;;22	}		
	00003e	b005	ADD	sp, sp, #0x14
32.	000040	bdf0	POP	{r4-r7, pc}

References

- [1] ARM Ltd., “ARM Compiler 5 Documentation,” [Online]. Available: <https://developer.arm.com/products/software-development-tools/compilers/arm-compiler/docs/version-5>, 2020.
- [2] ARM Ltd., *ARM Compiler v5.06 for μVision Version 5: armcc User Guide*, 2016.
- [3] ARM Ltd., *ARM Compiler v5.06 for μVision Version 5: armasm User Guide*, 2016.
- [4] Arm Ltd., *ARMv6-M Architecture Reference Manual*, DDI 0419D, 2017.
- [5] ARM Ltd., *ARM Compiler v5.06 for μVision Version 5: armlink User Guide*, 2016.

